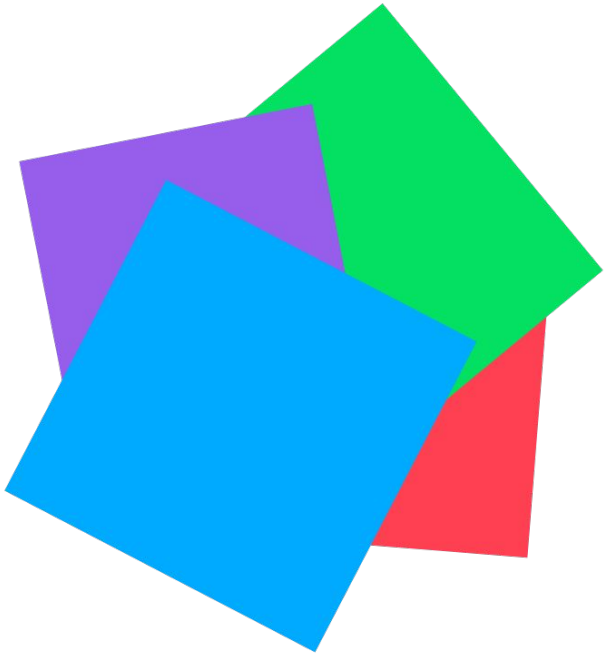
Паттерны программирования



Avito Start

Результаты обучения

1. Различать три группы паттернов

2. Выбирать паттерны и идиомы Go

3. Находить антипаттерны и объяснять цену абстракций



Что такое паттерн

01 **Повторяющаяся задача**

Стоимость доставки меняется независимо от оформления заказа.

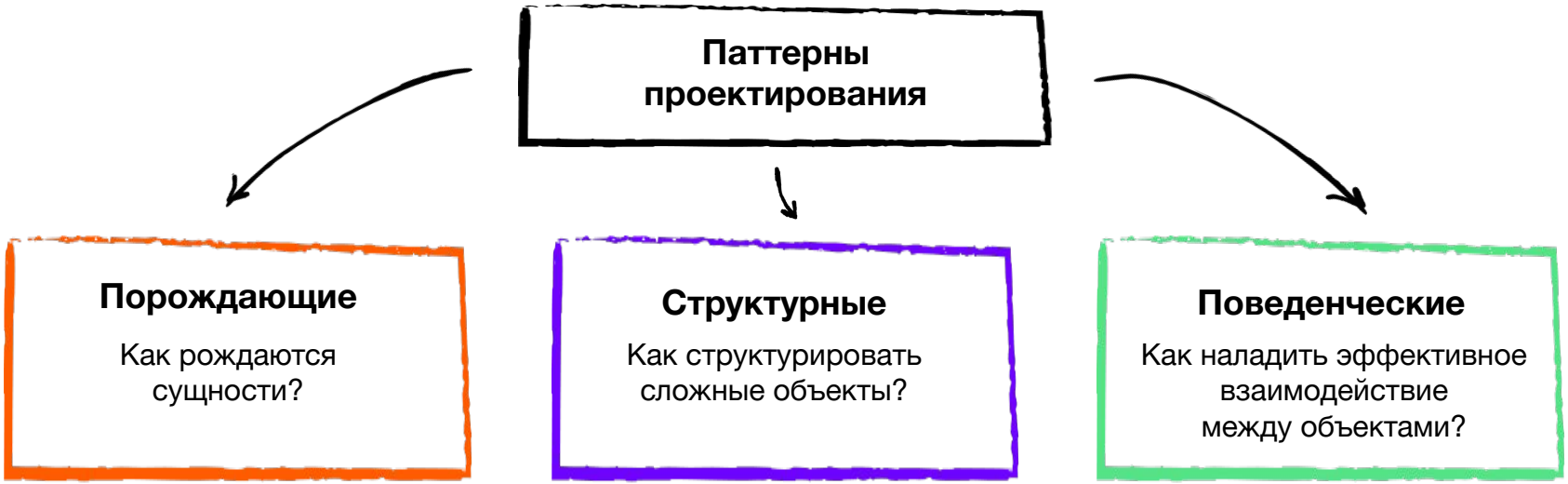
02 **Известная схема решения**

Выделяем расчёт итоговой цены и задаём точку подстановки алгоритма.

03 **Условия и последствия**

Новый вариант доставки добавить проще, но появляется контракт расчёта.

Три группы паттернов



Идиоматичный Go

01 **Функции как значения**

Расчёт доставки удобно передать как PriceFunc.

02 **Маленькие интерфейсы**

OrderStore требует только Save(Order) error.

03 **Композиция**

LoggedOrders оборачивает хранилище заказов.

04 **Простая конструкция по умолчанию**

Для фиксированного меню достаточно понятного switch.

Один сценарий: заказ пиццы

```
func Place(o Order, mode string) error {
    switch mode {
    case "pickup":
        o.Total = o.Base
    case "courier":
        o.Total = o.Base + 10_000
    }
    // Здесь же: оплата, логи и уведомления.
    return save(o)
}
```

Новый способ доставки увеличивает одну функцию

Strategy: расчёт доставки

01 **Задача**

При самовывозе доплаты нет. Курьер добавляет 100 рублей к сумме пицц.

02 **Решение**

Передаём расчёт итоговой цены как функцию. Оформление заказа использует общий контракт.

03 **Цена**

Точка сборки выбирает вариант. Для одного способа доставки отдельный контракт может быть лишним.

Strategy в заказе пиццы

```
type PriceFunc func(base int) int

func Pickup(base int) int { return base }
func Courier(base int) int { return base + 10_000 }

func Price(o *Order, calc PriceFunc) {
    o.Total = calc(o.Base)
}

Price(&order, Courier)
```

Новый способ доставки добавляет отдельную функцию

Factory: выбор рецепта

01 **Задача**

По названию пиццы нужно создать «Маргариту» или «Пепперони».

02 **Решение**

NewRecipe выбирает тип рецепта. Клиент использует общий метод BasePrice.

03 **Граница**

Это простая фабрика Go. Неизвестное название возвращает ошибку.

Фабрика рецептов

```go
type Recipe interface { BasePrice() int }

func NewRecipe(name string) (Recipe, error) {
    switch name {
    case "margherita":
        return Margherita{}, nil
    case "pepperoni":
        return Pepperoni{}, nil
    default:
        return nil, fmt.Errorf("unknown pizza: %s", name)
    }
}
```

Меню выбирает рецепт в одном месте

Builder: сборка пиццы

01 **Задача**

Гость выбирает размер и дополнительные ингредиенты. Готовая пицца должна быть допустимой.

02 **Решение**

Builder накапливает выбор. Build проверяет параметры перед выдачей результата.

03 **Цена**

Для фиксированной пиццы из меню обычно достаточно рецепта и конструктора.

Builder с проверкой размера

```
type Pizza struct { Size int; ExtraCheese bool }
type Builder struct { pizza Pizza }

func (b *Builder) Size(cm int) *Builder {
    b.pizza.Size = cm
    return b
}
func (b *Builder) Build() (Pizza, error) {
    if b.pizza.Size != 25 && b.pizza.Size != 30 {
        return Pizza{}, errors.New("invalid size")
    }
    return b.pizza, nil
}
```

pizza, err := new(Builder).Size(30).Build()

Как добавить доставку Express, не меняя функцию Price?

Adapter: подключение оплаты

01 **Задача**

Пиццерия вызывает Pay(копейки), а SDK банка ожидает Charge(amount, currency).

02 **Решение**

Адаптер переводит контракт приложения в вызов SDK и задаёт валюту RUB.

03 **Цена**

Адаптер должен сохранить ошибки и правильно передать денежные единицы.

Adapter для платёжного SDK

```
type Payment interface { Pay(kopecks int) error }
type BankSDK interface {
    Charge(amount int, currency string) error
}
type PaymentAdapter struct { bank BankSDK }

func (a PaymentAdapter) Pay(kopecks int) error {
    return a.bank.Charge(kopecks, "RUB")
}
```

Пиццерия видит Payment и не зависит от API банка

Decorator: журналирование заказа

01 **Задача**

Нужно измерять время сохранения заказа, не меняя его хранилище.

02 **Решение**

LoggedOrders реализует тот же Save и вызывает вложенный OrderStore.

03 **Цена**

Порядок обёрток влияет на измерения. Ошибка Save должна дойти до вызывающего кода.

Decorator для заказов

```
type OrderStore interface { Save(Order) error }
type LoggedOrders struct { next OrderStore }

func (s LoggedOrders) Save(o Order) error {
    started := time.Now()
    err := s.next.Save(o)
    log.Printf("save order duration=%s err=%v",
        time.Since(started), err)
    return err
}
```

Обёртка сохраняет контракт и исходную ошибку Save

Observer: пицца готова

01 **Задача**

На готовность заказа реагируют экран выдачи и уведомление гостю.

02 **Решение**

Кухня публикует OrderReady. Подписчики получают номер заказа через общий контракт.

03 **Контракт урока**

Вызовы синхронные. Регистрация завершается заранее. Медленный подписчик задерживает кухню.

Observer для готового заказа

```
type OrderReady struct { OrderID int64 }
type Listener func(OrderReady)
type Bus struct { listeners []Listener }

func (b *Bus) Subscribe(fn Listener) {
    b.listeners = append(b.listeners, fn)
}
func (b *Bus) Publish(e OrderReady) {
    for _, fn := range b.listeners {
        fn(e)
    }
}
```

Экран выдачи и уведомление подписываются на OrderReady

Чем Adapter отличается от Decorator?

Паттерны в стандартной библиотеке

01 **Классические паттерны**

HandlerFunc адаптирует функцию. HTTP middleware оборачивает обработчик.

02 **Идиомы Go**

Конструкторы передают зависимости явно. defer связывает ресурс с областью функции.

03 **Небольшие контракты**

Reader и Writer позволяют собирать обработку чека из готовых частей.

Adapter: http.HandlerFunc

```
func orderStatus(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")
    if _, err := io.WriteString(w, "Пицца готова"); err != nil {
        log.Print(err)
    }
}

var handler http.Handler = http.HandlerFunc(orderStatus)
```

Обычная функция становится http.Handler

Decorator: HTTP middleware

```
func WithTiming(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter,
        r *http.Request) {
        started := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("order request duration=%s",
            time.Since(started))
    })
}

handler := WithTiming(http.HandlerFunc(orderStatus))
```

Обёртка сохраняет Handler и вызывает next один раз

Композиция Reader и Writer

01 **Чек как поток**

Функция печати получает io.Writer и возвращает ошибку записи.

02 **Подстановка приёмника**

В тесте используем bytes.Buffer. В приложении — файл или другой Writer.

03 **Композиция чтения**

io.LimitReader ограничивает число байтов. io.Copy переносит поток в Writer.

Чек через io.Writer

```
func PrintReceipt(w io.Writer, o Order) error {
    _, err := fmt.Fprintf(w, "Order #%d: %d kop\n",
        o.ID, o.Total)
    return err
}

var receipt bytes.Buffer
if err := PrintReceipt(&receipt, order); err != nil {
    log.Print(err)
}
```

Один контракт для чека в памяти и в файле

Композиция потока чека

```go
func ReceiptPreview(w io.Writer, receipt string) error {
    source := strings.NewReader(receipt)
    preview := io.LimitReader(source, 80)
    _, err := io.Copy(w, preview)
    return err
}
```

Источник, ограничитель и приёмник независимы

Результат: роли в пиццерии

01 **Создание и расчёт**

NewRecipe выбирает рецепт. Builder собирает пиццу. PriceFunc считает итог с доставкой.

02 **Оплата и сохранение**

PaymentAdapter подключает банк. LoggedOrders измеряет сохранение заказа.

03 **Готовность**

Кухня публикует OrderReady после приготовления. Bus оповещает подписчиков.

God Object в пиццерии

01 **Признаки**

PizzaService хранит меню, считает доставку, списывает оплату, управляет печью и отправляет SMS.

02 **Последствия**

Для теста цены приходится собирать банк, кухню и хранилище.

03 **Рефакторинг**

Выделяем расчёт цены, оплату и приготовление по причинам изменения. Сценарий координирует шаги.

Spaghetti Code в заказе

01 **Признаки**

Вложенные if смешивают скидку, оплату, запуск печи и отмену заказа.

02 **Последствия**

Трудно понять, может ли пицца уйти в печь после отказа оплаты.

03 **Рефакторинг**

Фиксируем поведение тестами. Выделяем явные шаги и ранний возврат при ошибке.

Premature Abstraction

01 **Признаки**

Универсальный FoodEngine с плагинами ради двух пицц из одного меню.

02 **Последствия**

Чтобы добавить начинку, нужно изучить DSL и несколько слоёв конфигурации.

03 **Рефакторинг**

Возвращаем явные рецепты. Общий контракт вводим после появления устойчивой потребности.

Выбор паттерна по задаче

01 **Рецепт, сборка или расчёт**

Factory выбирает рецепт. Builder собирает пиццу. Strategy считает доставку.

02 **Интеграции и реакции**

Adapter подключает банк. Decorator журналирует заказ. Observer сообщает о готовности.

03 **Стабильное меню**

Для одной пиццы и самовывоза достаточно обычных функций и конкретных типов.

Какой антипаттерн иллюстрирует FoodEngine с плагинами ради двух пицц?

Итоги занятия

01 Классические паттерны адаптируются к средствам Go

02 HandlerFunc и middleware показывают Adapter и Decorator

03 io.Writer позволяет печатать чек в разные приёмники

04 Конструктор и defer делают зависимости и ресурсы явными

05 **Практика: доставка, HTTP и чек без лишних абстракций**

Материалы для продолжения

01 **Основы Go**

Effective Go: функции, интерфейсы и композиция
go.dev/doc/effective_go

02 **Классические паттерны**

Gamma, Helm, Johnson, Vlissides. Design Patterns
Каталог и условия применения

03 **Следующие занятия**

Конкурентные паттерны, затем паттерны микросервисов


Q&A
time